```python
# Continue & break

#continue: will make it go to the next iteration

#break: stops the loop

samson = [34, 78, 20, 100, 220, 12, 28, 76]
# I want to print everything until 100
#34 78 20 100 ... break

for value in samson: #
  print(value)
  if value == 100:
    break
```

```
34
78
20
100
```

```python
samson = [34, 78, 20, 100, 220, 12, 28, 76]
# I want to print everything until 100
#34 78 20 100 ... break

for value in samson: #
  print(value) #34 78 20 100 220 12 28 76
  if value == 100:
    continue
```

```
34
78
20
100
220
12
28
76
```

```python
samson = [34, 78, 20, 100, 220, 12, 28, 76]
# We want t oskip 100

for value in samson:
  if value == 100: #220
    continue
  print(value) #34 78 20 220 12 28 76
```

```
34
78
20
220
12
```

```
28
76
```

- Break stops the entire loop
- continue proceeds to the next iteration ignoring everything below the continue

Guess the number game

1. we make our user guess a number between 1 and 100
2. We stop and say congratulations when he gets it right
3. we proceed asking until he gets it

```
# are we going to use for looop or while loop? and why?

# while loop: because we don't know how many times it is going to
iterate

#what's our conditon

target = 34
guess = int(input("Guess a number between 1 and 100: "))

if guess == target:
  print("Congratulations, you got it")
else:
  print("please try again")

Guess a number between 1 and 100: 87
please try again

#let's use while loopo
target = 34
guess = int(input("guess a number between 1 and 100: "))
while guess != target:
  guess = int(input("please try again a number between 1 and 100: "))
print("Congratulations, you have got the number!")

guess a number between 1 and 100: 87
please try again a number between 1 and 100: 23
please try again a number between 1 and 100: 34
Congratulations, you have got the number!

#let's use while loop with break
target = 34
guess = int(input("guess a number between 1 and 100: "))
while True:
  if guess == target:
    break
  guess = int(input("please try again a number between 1 and 100: "))
print("Congratulations, you have got the number!")
```

```
guess a number between 1 and 100: 45
please try again a number between 1 and 100: 23
please try again a number between 1 and 100: 95
please try again a number between 1 and 100: 34
Congratulations, you have got the number!
```

Nested Loops(loop inside another loop)

```
Make sure the variables for the loops are different

# please trace the following code, one step at a time
for i in range(3): # 0, 1, 2
#i = 2
    for j in range(3): # 0, 1, 2
    # j = 2
        print(i,j) # 0 0, 0 1, 0 2, 1 0, 1 1, 1 2, 2 0, 2 1, 2 2

0 0
0 1
0 2
1 0
1 1
1 2
2 0
2 1
2 2

# Please walk through the loop one step at a time
for i in range(3): # 0, 1, 2
#i = 2
    for j in range(i): #range(0) empty
    #range(1) = 0
    #range(2) = 0, 1
        print(i,j) # 1 0, 2 0, 2 1

1 0
2 0
2 1
```

Remember that we can put a list inside another list and the indexing of such list requires usage of `nested loop`

For example: we have our list named "nums"

```
nums = [[1, 5, 18, 8], [23, 7, 9, 19], [2, 3, 28, 10]]
```

1. if we say nums[0] we get [1, 5, 18, 8] as a list
2. if we want 18, we need to index one more time like:

```
nums[0][2] #this means go to the list index 0 and get [1, 5, 18, 8]
and then go to this list index 3 which is 18

nums = [[1, 5, 18, 8], [23, 7, 9, 19], [2, 3, 28, 10]]
nums[2][0] #what do you expect?

2

nums[2][1]

3
```

Now let's write a code to print every item in the list one by one like:

```
1
5
18
8
23
7
9
19
2
3
28
10
```

Remember to use nested loop because we need two indices

```
#your code here
nums = [[1, 5, 18, 8], [23, 7, 9, 19], [2, 3, 28, 10]]
for i in nums:
  print(i) #i itself is a list

[1, 5, 18, 8]
[23, 7, 9, 19]
[2, 3, 28, 10]

i = [1, 5, 18, 8]
for v in i:
  print(v)

1
5
18
8

#your code here
nums = [[1, 5, 18, 8], [23, 7, 9, 19], [2, 3, 28, 10]]
for i in nums: # i = [1, 5, 18, 8]
```

```
  for j in i: #
    print(j)
```

```
1
5
18
8
23
7
9
19
2
3
28
10
```

Using 'else' clause in loops

- When the loop condition of `"for" or "while" statement fails then code part in "else" is executed`
- **`If a break statement is executed inside the for loop then the "else" part is skipped.`**
- **`Note that the "else" part is executed even if there is a continue statement.`**

```
# Prints out 0,1,2,3,4 and then it prints "count value reached 5"

count=0
while(count<5):
    print(count)
    count +=1 # 5
else:
    print(f"count value reached {count}")
```

```
0
1
2
3
4
count value reached 5
```

```
# Prints out 1,2,3,4
for i in range(1, 10):
```

```
    if(i%5==0):
        break
    print(i)
else:
    print("this is not printed because for loop is terminated because
of break but not due to fail in condition")
```

```
1
2
3
4
```

Trace the following code and guess the output

```
for i in range(3):
    for j in range(3):
        print(i,j)
```

Functions

A function is a block of code which only runs when it is called.

- Convenient way to divide our code into useful blocks
- Allowing us to order our code
- Make it more readable
- Reuse it and save some time

Functions in python are defined using the block keyword **"def"**, followed with the function's name

```
    def function_name(arguments if any):
        component of the function

#declaring a function named "Hello"
def Hello(): #it has no arguments
  print("Hello from a function") #this is what hte function is doing,
printing

Hello() #calling my function
```

```
Hello from a function
```

```
def call_name():
    print('Hello,Alem')
call_name()
```

```
Hello,Alem
```

Remember function runs only when it is called

```
# let's call our function
Hello() # we are calling our function

Hello from a function

call_name()

Hello,Alem
```

Be mindful of Indentation for a block

```
The function body must be indented
Like if conditions and loops, functions are put in a block,
every part inside the function should be indented
```

e.g1.

```
def function_name():
print("hello")   # this is wrong because there is no indentation (look at the error printed)
function_name()

def function_name():
    print('hello')  # this is correct
```

This is correct

Types of Functions

There are two types of functions in Python:

1. **Built-in functions**:

These are the functions that are already built-in in Python such as:

- help() to ask for help,
- min() to get the minimum value,
- print() to print an object to the terminal,
- range() to get numbers in a given range.

```
# getting the absolute value
abs(-10)

10

# getting input from user
input("hello, please insert your name: ")

hello, please insert your name: winner
```

```
{"type":"string"}

# finding maximum and minimum
l = [26, -90, 4]
max(l), min(l)

(26, -90)
```

2. **User-Defined Functions (UDFs):**

These are the functions **`created by the user`** according to their programming needs.

Let's create a function that takes two numbers as arguments and adds them and returns the sum

```
def sum(marefia, habtamu):   # This is a user defined function that
takes two arguments
    return marefia + habtamu # and adds them up.
```

`Rememeber again inorder to use a function, we need to call it,` so let's call it

```
sum(27, 5) #takes two numbers a =2 and b =5 inthis case and returns
their sum which is 7

32

def sum_with_print(marefia, habtamu):   # This is a user defined
function that takes two arguments
    print(marefia + habtamu) # and adds them up.

sum_with_print(20, 30)  + 10

50

---------------------------------------------------------------------------
-----
TypeError                                 Traceback (most recent call
last)
/tmp/ipykernel_170/2777123366.py in <cell line: 0>()
----> 1 sum_with_print(20, 30)  + 10

TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'

sum(20, 30) + 10

60
```

Every function takes 0 or more arguments

- Information can be passed into functions as arguments.

- Arguments are specified after the function name, inside the parentheses.
- You can add as many arguments as you want, just separate them with a comma.

```
def function(argument):  # as you can see, our function 'func' has
argument named 'argument'
    print(argument)   # the job of this function is to print the
argument

function(28)

28

# Let's call our function func to see what it does
function()  # if we call it like this, we get an error because func
has argument in it

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
/tmp/ipykernel_170/3535954218.py in <cell line: 0>()
      1 # Let's call our function func to see what it does
----> 2 function()  # if we call it like this, we get an error because
func has argument in it

TypeError: function() missing 1 required positional argument:
'argument'

function('hello') # now we are correct as we specified our argument

hello

function(1) # whatever we put for the arguemnt will be printed

1
```

Let's create a function which multiplies a number by two and prints it

let's name our function as `"multiplyByTwo"`

```
def multiplyByTwo(n):
  print(n*2)

multiplyByTwo(20)

40

multiplyByTwo([2, 9, 8])

[2, 9, 8, 2, 9, 8]
```

```
multiplyByTwo("hello")
```

```
hellohello
```

```
def multiplyByTwo(number): #takes argument named number
    print(number*2) #doubles the argument and prints it

# Let's call it
multiplyByTwo(3) # so it should print 3*2 = 6
```

```
6
```

```
# Let's create a function that adds two numbers given in parament
# Let's name it "add"
def add(a, b):
    print(a + b)

add(2, 5)
print(add(4,3)+7) #causes an eerror due to print instead of return
```

```
7
7
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_170/2440727227.py in <cell line: 0>()
      1 add(2, 5)
----> 2 print(add(4,3)+7) #causes an eerror due to print instead of return

TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

```
def add(a, b):
    return a + b

add(2, 5)
print(add(4,3)+7)
```

```
14
```

```
# Let's create function that takes firstname and last name and print's them together
def fullName(firstName, lastName):
    print(firstName + " " + lastName)

# Let's call it
fullName("Marefia", "Adane")
```

```
Marefia Adane
```

```
# be careful on the number of arguments you have
fullName("John") # we get an error here b/c we need to arguments but
we gave only one
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
/tmp/ipykernel_170/1838761039.py in <cell line: 0>()
      1 # be careful on the number of arguments you have
----> 2 fullName("John") # we get an error here b/c we need to
arguments but we gave only one

TypeError: fullName() missing 1 required positional argument:
'lastName'
```

```
grade = 98
name = "Eyobed"
# Hey Eyobed your grade is 98
print(f"Hey {name} your grade is {grade}")
```

```
Hey Eyobed your grade is 98
```

```
# Another example
def my_function(username, greeting):
    print(f"Hello {username}, From My Function!, I wish you
{greeting}") # remember string formatting
```

```
my_function('Dave', "Happy birthday")
```

```
Hello Dave, From My Function!, I wish you Happy birthday
```

```
# functions can have code inside them
def function_with_body(x): #x=30
    y = x + 10
    print(y)
```

```
function_with_body(30)
```

```
40
```

Returning a value from function

```
use 'return' statement
```

```
# imagine we want the function to give us/return the sum of two
numbers
def sum(a, b):
    return a + b

sum(3,9)
```

```
12

print(sum(8,3)+4)

15

# function with more bodies
def function_bigBody(x): #x=10
    x += 5 #15
    x *= 3 #45
    x -= 2 #43
    x %= 100 #43
    return x

function_bigBody(10) # try to trace it before printing it

43
```

Anything after "return" will be ignored!! afer return the function stops running

```
def sum(a, b):
    return a + b
    print(b)   # this print statement is after the return statemnet,
so it will be ignored

sum(3,9) # it will just retun the sum, will not print 9

12

# to print 9, we should make it above the return statement
def sum(a, b):
    print(b)  # now this one will be executed
    return a + b

sum(3, 9)

9

12
```

`Exercise`

1. write a function that takes 2 arguments x and y. then it swaps them.

```
# x = 10 ... milk
# y = "hello"  .... tea
#after swapping
# x = "hello" ... tea
# y = 10 ... milk

#we need third variablr.. z
# z = x  .. you put the milk on another cup
```

```
# x = y .. you put the tea on the x
# y = z .. you put the milk on y
def swap(x, y):
  z = x
  x = y
  y = z
  return x, y

swap(20, 22)
```

```
(22, 20)
```

1. Add a function named list_benefits() that returns the following list of strings: "you", "are", "amazingly", "made"

1. Add a function named build_sentence(info) which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"

1. Write a function that takes a single number as an argument and return the list of the first 5 multiples of the number.

1. What is the output of the following function?

```
def myFunction(a):
    print(10 * a)
    return a
a = 5
myFunction(a) # what do we get?
```

1. What is the output of the following?

```
def myfunction(a):
    print(10 * a)
    return a
    print('My name is Alem')
a = 5
myfunction(a) # what do we get?
```

1. What is wrong with the following code?

```
def myfunction(a):
    print(10*a)
    return a
    print('My name is Alem')
a = 5
z = myfunction(a)
print(z)
```

1. What is wrong with the following code?

```
def myfunction(a):
    print(10*a)
return a
print('My name is Alem')
a = 5
z = myfunction(a)
print(z)
```

1. write a function that takes `score as an argument` and `returns True if it it's greater than 50 or False if it is not`.

```

```

1. What is the difference between the following 2 functions.

```
def loopFunction(x):
    for i in range(x):
        print(i)

def loopFunction2(x):
    for i in x:
        print(i)
```

1. Write a function that takes string as an argument and prints out every character of the string separately. e.g. func("Ethiopia") .. the name of the function is func, it is taking the string "Ethiopia" the output will be:

```
    E
    t
    h
    i
    o
    p
```

```
    i
    a
```